

One word

"A particular way of **organizing data set** into **computer systems** so that its **access** is **efficient**, usually at the cost of sublinear complexity of **amortized time**."

A specific way of organizing a data set into a computer system so that the data can be accessed efficiently, usually at the cost of amortizing the time complexity below linear.

Total Ordered Data Set

A **Total Ordered Data Set** is one in which every pair of elements can be compared with each other in a way that satisfies the mathematical definition of a "total order relationship". A total order relationship ensures that the data can be arranged in a defined order, thus supporting efficient retrieval, sorting, and other operations.

I. Total Order Relationship (Total Order) Definition

Let there be a relation R on the set S , and R is said to be a **Total Order** relation on S if the following four conditions are satisfied:

- 1. **Reflexive:**
For any $a \in S$, there are $a \leq a$
- 2. **Antisymmetric:**
If $a \leq b$ and $b \leq a$ then $a = b$
- 3. **Transitive:**
If $a \leq b$ and $b \leq c$, then $a \leq c$
- 4. **Totality or Comparability:**
For any $a, b \in S$, there must be $a \leq b$ or $b \leq a$.

II. Meaning of Total Ordered Data Set

It is a class of data sets whose elements can satisfy the "full order" relationship described above, so that they can always be compared and uniquely ordered.

Examples:

Dataset	Full-order basis
Student Number Collection	Sort by Numeric Size
Name Collection	Sort by Pinyin or Dictionary Order
Movie Titles Collection	Alphabetical Order
List of timestamps	Sort by early or late

Example:

School number: 202101, 202103, 202102 → can be sorted as: 202101 < 202102 < 202103

Name: Alice, Bob, Charlie → can be sorted as Alice < Bob < Charlie

III. Uses of the Total Ordered Data Set

Because the data can be sorted, many classical algorithms can be applied efficiently:

- **Binary Search:** time complexity $O(\log n)$
- **Sorting algorithms:** fast sort, merge sort, etc.
- **Ordered index structures:** e.g., B+ trees, red-black trees, etc. are constructed on the basis of the full-order property.

IV. Practical implications (in information systems/databases)

In databases, information retrieval systems, full-sequence data can be:

- Indexing (e.g., B-tree indexing by time or dictionary order)
- Supports range queries (e.g., to find students with student numbers in [202100, 202199])
- Efficient support for sorted displays (e.g., output in ascending alphabetical order by name)

V. Summary

Characteristics	Description
Comparability	Any two elements can be compared
Stability	Satisfaction of mathematical full order relations
Algorithm Support	Efficient Search and Sorting
Example	Student number, name, date, number, etc.

Total Ordered Data Set is a data set that is regularly structured, has well-defined relationships, supports comparison and ordering, and is one of the foundations of computation and algorithms.

Succinct Data Structures

Succinct data structures are data structures that **support efficient operations** while **approaching information-theoretic optimal spaces**. Their core goal is:

Use as little space as possible (close to the theoretical lower bound) while still keeping operations such as accessing, querying, and modifying data with low time complexity (usually constant or logarithmic time).

I. Background and motivation

In traditional data structures, we tend to pursue **time complexity minimization** without paying special attention to space usage efficiency. However, space becomes extremely important in the following scenarios:

- **Large-scale data processing** (e.g., Web charts, genome sequences, compressed text indexing)
- **Memory-constrained devices** (e.g., embedded systems, mobile devices)
- **Cache optimization and data compression**

Therefore, researchers have proposed a class of "**spatially compact and powerful**" data structures -- succinct data structures.

II. Definitions (mathematical perspective)One word

Double-ended Priority Queues

"A particular way of **organizing data set** into **computer systems** so that its **access** is **efficient**, usually at the cost of sublinear complexity of **amortized time**."

A specific way of organizing a data set into a computer system so that the data can be accessed efficiently, usually at the cost of amortizing the time complexity below linear.

The **Double-ended Priority Queue (DEPQ)** is a data structure that supports simultaneous prioritization operations** at **both ends, and is capable of efficiently performing the following operations:

Total Ordered Data Set

I. Basic definitions

A **Total Ordered Data Set** is one in which every pair of elements can be compared with each other in a way that satisfies the mathematical definition of a "total order relationship". A total order relationship ensures that the data can be arranged in a defined order, thus supporting efficient retrieval, sorting, and other operations.

Double-ended Priority Queue (DEPQ) is a special type of queue that not only supports:

- **Insert element**

I. Total Order Relationship (Total Order) Definition

- **Deletion of the smallest element (Delete-Min)**

Let there be a relation R on the set S , and R is said to be a **Total Order** relation on S if the following four conditions are satisfied:

- **Delete-Max**

1. **Reflexive:**

For any $a \in S$, there are $a \leq a$

2. **Antisymmetric:**

If $a \leq b$ and $b \leq a$ then $a = b$

3. **Transitive:**

If $a \leq b$ and $b \leq c$, then $a \leq c$

4. **Totality or Comparability:**

For any $a, b \in S$, there must be $a \leq b$ or $b \leq a$.

Also support:

- **Find-Minimum (Find-Min)**

II. Meaning of Total Ordered Data Set

- **Find-Max**

It is a class of data sets whose elements can satisfy the "full order" relationship described above, so that they can always be compared and uniquely ordered.

In contrast to a normal priority queue, such as a heap, DEPQ allows you to operate on both the **maximum and minimum** values of a priority.

Examples:

II. Supported operations

| Dataset | Full-order basis |

| ----- | ----- |

| Student Number Collection | Sort by Numeric Size |

| Name Collection | Sort by Pinyin or Dictionary Order |

| Movie Titles Collection | Alphabetical Order |

| List of timestamps | Sort by early or late |

| Operation Name | Description | Time Complexity (implementation dependent) |

Example:

| ----- | ----- | ----- |

``

School number: 202101, 202103, 202102 → can be sorted as: 202101 < 202102 < 202103

Name: Alice, Bob, Charlie → can be sorted as Alice < Bob < Charlie

``

| **insert(x)** | insert element **x** | **O(log n)** |

| **find-min()** | find smallest element | **O(1) ~ O(log n)** |

III. Uses of the Total Ordered Data Set

| **find-max()** | find-max-elements | **O(1) ~ O(log n)** |

Because the data can be sorted, many classical algorithms can be applied efficiently:

| `delete-min()` | deletes and returns the smallest element | $O(\log n)$ |

- **Binary Search:** time complexity $O(\log n)$
- **Sorting algorithms:** fast sort, merge sort, etc.
- **Ordered index structures:** e.g., B+ trees, red-black trees, etc. are constructed on the basis of the full-order property.

| `delete-max()` | deletes and returns the largest element | $O(\log n)$ |

III. Common implementations

IV. Practical implications (in information systems/databases)

1. Min-Max Heap

In databases, information retrieval systems, full-sequence data can be:

- A special kind of heap in which each layer alternates between a minimum and a maximum layer.
 - Indexing (e.g., B-tree indexing by time or dictionary order)
 - Supports range queries (e.g., to find students with student numbers in [202100, 202199])
 - Efficient support for sorted displays (e.g., output in ascending alphabetical order by name)
 - The root is the smallest element, its child is the largest element, its grandchild is the smallest element, and so on.
-

- **Time complexity:**

V. Summary

- `insert`: $O(\log n)$

| Characteristics | Description |

| ----- | ----- |

| Comparability | Any two elements can be compared |

| Stability | Satisfaction of mathematical full order relations |

| Algorithm Support | Efficient Search and Sorting |

| Example | Student number, name, date, number, etc. |

- `delete-min` / `delete-max`: $O(\log n)$

Total Ordered Data Set is a data set that is regularly structured, has well-defined relationships, supports comparison and ordering, and is one of the foundations of computation and algorithms.

- `find-min` / `find-max`: $O(1)$
-

2. Two heaps (Min Heap + Max Heap)

Succinct Data Structures

- A minimum and a maximum heap are used to maintain the minimum and maximum values, respectively, and a mapping is used to synchronize the positions of the elements in the two heaps.

Succinct data structures are data structures that **support efficient operations** while **approaching information-theoretic optimal spaces**. Their core goal is:

- Highly complex to implement, but flexible to maintain.

Use as little space as possible (close to the theoretical lower bound) while still keeping operations such as accessing, querying, and modifying data with low time complexity (usually constant or logarithmic time).

3. Balanced binary search trees (e.g., red-black, AVL)

- It also supports fast insertion, deletion, minimum and maximum value lookup.

I. Background and motivation

- The time complexity is $O(\log n)$ for all of them.

In traditional data structures, we tend to pursue **time complexity minimization** without paying special attention to space usage efficiency. However, space becomes extremely important in the following scenarios:

- Sequential statistics (e.g., kth smallest, kth largest) can be supported.
- **Large-scale data processing** (e.g., Web charts, genome sequences, compressed text indexing)
- **Memory-constrained devices** (e.g., embedded systems, mobile devices)
- **Cache optimization and data compression**
- **Scheduling systems**: e.g., finding the optimal (largest) or the worst (smallest) task in task allocation.

Therefore, researchers have proposed a class of "**spatially compact and powerful**" data structures -- succinct data structures.

- **Financial system**: dynamic maintenance of maximum and minimum bids.

- **Multi-window prioritization**: e.g., double-ended constraints on resource scheduling in operating systems.

II. Definitions (mathematical perspective)

- **Sliding window problem**: dynamic maintenance of max-min values (combined with double-ended queues or tree structures).

Let the set of objects of a certain class be S , whose **information entropy lower bound** is the minimum number of bits required to store objects of that class, denoted $I(n)$.

External Sorting: External Sorting. Improve Run Gen. Improve Run Merging

Then a **succinct data structure** is in space complexity:

Red and black trees

$I(n) + o(I(n))$ bits

A **Red-Black Tree** is a **self-balancing Binary Search Tree (BST)**, which ensures that the tree is always "approximately balanced" after performing insertions, deletions, etc. by adding a color bit (red or black) to each node. By adding a color bit (red or black) to each node, it ensures that the tree is always "approximately balanced" after performing insertions, deletions, etc., thus guaranteeing that the time complexity of all basic operations is **$O(\log n)$** .

Under this, data structures that support efficient operations (e.g., querying, modifying, inserting, etc.) are also available.

I. Basic characteristics of red and black trees

where $o(I(n))$ is a low-order function much smaller than $I(n)$, such as $\log n$ or $\log \log n$.

The red-black tree is based on the binary lookup tree, by introducing "color" and strict rules to control the height of the tree.

Each node contains:

III. Typical examples

- `key`: key value

1. Bit Vector + Rank/Select

- `color`: color (Red Red or Black Black)

For a bitmap $B \in \{0, 1\}^n$ of length n , the succinct data structure can be in:

- `left`: left child node
- Space: $n + o(n)$ bit
- Operation:
- **rank(i)**: query how many 1s there are in $B[1..i]$
- **select(k)**: query for the location of the k th 1 occurrence
- `right`: right child node

This type of structure is widely used for text indexing (e.g., FM-index) and graph compression.

- `parent`: parent node
-

II. Five Properties of the Red-Black Tree (The Key to Maintaining Balance)

2. Balanced Parentheses

- 1. **Each node is either red or black.**
Used to represent tree or expression structures (e.g., XML/JSON structured trees) that can be supported in near-theoretical space:
 - 2. **The root node is black.**
 - Find matching brackets (match subtree)
 - Query Parent-Child Node Relationships
 - Subtree size calculation and other operations
 - 3. **Each leaf node (NIL node, null node) is black.**
-
- 4. **Red nodes cannot appear consecutively (children of red nodes must be black).**

3. Succinct Trees

- 5. **The path from any node to all of its descendant leaf nodes contains the same number of black nodes (black-height consistency).**
Used to store tree structures such as:

III. Equilibrium properties of red-black trees

- The optimal encoding of an ordered tree of n nodes requires about $2n$ bit
- Using structures such as Jacobson/Farzan-Munro can support queries for parents, children, LCAs (Least Common Ancestors), etc. within $2n + o(n)$ bit, with a time complexity of $O(1)$ or $O(\log n)$

These rules ensure the following properties:

-
- The longest path of a red-black tree will not be more than twice the shortest path.

IV. Comparison with other concepts

- The height of the tree is at most $2 \lceil \log(n + 1) \rceil$, which maintains the **logarithmic height** and thus achieves $O(\log n)$ operational efficiency.

Types of Data Structures	Space Usage	Operational Efficiency	Features
-----	-----	-----	-----
traditional structures (e.g., chained lists, trees)	large (often with lots of pointers)	fast	space-wasting
Compressed Data Structures	Extremely Small Space	Doesn't necessarily support queries	Mostly used for compressed storage
Succinct Data Structure	Near-optimal (information-theoretic lower bound)	Fast (usually $O(1)$)	Both compressed and manipulable

IV. Time complexity of common operations on red-black trees

| operations | time complexity | description |

V. Application scenarios

| --- | ----- | ----- |

- **Text compression and full-text indexing (e.g., FM-index)**
- **:: Large-scale graph structures (Web graphs, social network graphs)**
- **Genome storage and querying (e.g. BWT structure)**
- **Compressed representation of XML/JSON trees**
- **Compressed Boolean matrix**

| Search | $O(\log n)$ | Similar to normal BST search |

| Insert | $O(\log n)$ | Adjust by color transformation and rotation after insertion |

VI. Summary

| Delete | $O(\log n)$ | Adjust after deletion by color transformation and rotation |

| Characteristics | Description |

| ----- | ----- |

| space complexity | approaching lower bound on theoretical information entropy |

| Operations | Support common query operations with low time complexity |

| Applications | Large Scale Data, High Compression Ratio Requirement Scenarios |

| Common Construction Methods | Bitmap + Auxiliary Structure based, Tree Encoding, etc. |

V. How do red and black trees stay balanced?

Simple data structures strike a high balance between "**compressed storage + manipulability**", and are a very important class of structures in modern computing.

Key operations: Rotate + color transformations

heap-like notation for a binary tree

- **Left Rotate** and **Right Rotate**: used to restructure the tree.

A **Heap-like notation for a binary tree** is a way to represent a **Complete Binary Tree** by an array (usually starting at subscript 1), which is widely used in heap structures, but can also be used for sequential storage representation of other binary trees.

- **Color Flipping (Recoloring)**: adjusts the color of a node to satisfy the properties of a red-black tree.

After inserting or deleting a node, if the nature of the red-black tree is violated, the balance is restored by "Rotation + Color Transformation".

I. Basic ideas

VI. Application of red-black trees

In a **heap-like notation**, the nodes of a binary tree are stored in an array in **Level-order traversal** order, numbered starting at **1** (or sometimes 0, but a typical heap is more intuitive starting at 1). Red-black trees are widely used in many standard libraries and systems, for example: For a node numbered `i`, the relationship is as follows:

- C++ STL for `map` and `set`

Relationships	Formulas	Meanings
---------------	----------	----------

Parent	`[i / 2]`	The parent subscript of the i-th node. For example, if i=4, the parent is 2.
Left child node	`2 × i`	The left child subscript of the i-th node. For example, if i=2, the left child is 4.
Right child node	`2 × i + 1`	The right child subscript of the i-th node. For example, if i=2, the right child is 5.

- Java's `TreeMap` and `TreeSet`

- Process scheduling, memory management and other modules in the Linux kernel

II. Examples

- Some data structures in database indexing systems (e.g., PostgreSQL)
Consider the following binary tree (with the largest heap as an example):

VII. Comparison with other balanced trees

```
  ``
50
 / \
30 20
 / \ /
10 15 18
  ``
```

| Tree Types | Balance | Insertion Deletion Consideration | Common Scenarios

Stored as an array (starting at subscript 1) with heap-like notation:

| ----- | ----- | ----- | ----- |

subscript (i)	numeric
---------------	---------

| 1 | 50 |

| 2 | 30 |

| 3 | 20 |

| 4 | 10 |

| 5 | 15 |

| 6 | 18 |

| Red-Black Tree | Medium | $O(\log n)$, stable performance | Mostly used in standard libraries and systems programming |

Relationship Validation:

| AVL trees | stricter | slightly more costly insertions/deletions | scenarios with frequent lookups and few insertions |

- The left child of node 1 is at subscript 2 (30) and the right child is at subscript 3 (20)

| Treap (Random Trees) | Probability Balance | Insertion Deletion Average $O(\log n)$ | Common in Competition or Research |

- The left child of node 2 is at subscript 4 (10) and the right child is at subscript 5 (15)
| B-Tree/B+-Tree | Multiplexed Balanced Tree for Disk | Insertion Deletion More Complex | Databases and File Systems |
- The left child of node 3 is at subscript 6 (18)

III. Advantages and limitations

✓ Advantages

- **No need for pointers**, implement tree structures with arrays
- For **complete binary trees** such as heaps
- **Parent-child relationships are computationally simple**, $O(1)$ Access

✗ Limitations

- Wasted array space if used for **non-complete binary trees**
- Unsuitable for representing highly unbalanced trees (e.g., chained structures)

IV. Application scenarios

- **Heap implementation**
- Maximum Heap / Minimum Heap
- priority queue
- **Heap Sort**
- **Segment Tree**

- **Fenwick Tree**
- for certain memory compact structures such as heap structures in graph algorithms (e.g. Dijkstra)

V. Comparison with other encoding methods

| Encoding | Pointers or not | Space Efficiency | Applicable Structures |

| ----- | ----- | ----- | ----- |

| Heap-like notation | No | High | Complete/near-complete trees |

| Linked representations | Yes | Flexible | Arbitrary structures |

| Succinct representation | No (bit-coded) | Optimal | Large-scale static trees |

Point Query: Min/Max, kth element access

IV. Application scenarios

Min-Max Heap

What is Min-Max Heap?

Min-Max Heap is an array-based Complete Binary Tree that divides nodes into **Min layers** and **Max layers** according to their depth, as follows:

- **The root node is at the Min level (depth 0)**, indicating the minimum value of the whole tree.
- **** Its child is at the Max level (depth 1)****, indicating the subtree maximum.
- **grandchild node back to the Min level (depth 2)**, denoting the subtree minimum.
- So alternately.

That is to say:

- **Even layers (0, 2, 4, ...)** are **Min layers** that maintain the minimum value;
- **Odd layers (1, 3, 5, ...)** are **Max layers** that maintain the maximum value.

Operation

| operations | time complexity | description |

| ----- | ----- | ----- |

| `findMin()` | $O(1)$ | root node |

| `findMax()` | $O(1)$ | greater of two children of root |

| `insert(x)` | $O(\log n)$ | inserts and "floats" to the correct position |

| `deleteMin()` | $O(\log n)$ | delete root node and "sink" |

| `deleteMax()` | $O(\log n)$ | delete max node and "sink" |

1. Node Hierarchy Determination

Given a node's subscript i (starting from 1) in an array with layers:

$\text{level}(i) = \lfloor \log_2(i) \rfloor$

Determines if it is at the Min level:

$\text{is_min_level}(i) = \text{level}(i) \% 2 == 0$

2. Insertion operations

- Inserts the new element into the end of the array;
- Determine which layer (Min or Max) it is on;
- Compare and float up to the correct position, following the rules of Min Heap or Max Heap, respectively.

3. Delete operations (lookups are similar, except that there is no delete operation)

deleteMin()

- Delete the root node (subscript 1);
- Replace with the last element of the array;
- Sinks downward, maintaining the Min-Max Heap nature.

deleteMax()

- Find the larger of the two children (subscripts 2 and 3) of the root;
- Delete it and replace it with the last element;
- Downward "sinking".

Dynamic sequential statistics

Common operations

1. OS-SELECT(i, S).

- This operation returns the i -th smallest element in the dynamic set S .
- For example, if $i=3$, then the operation will return the third smallest element in the set.

2. OS-RANK(x, S).

- This operation returns the rank (i.e., the position) of the element x after it has been sorted in the set S .
- The rank is the position of the element in the sorted set, e.g., if x is the second smallest element in the set, then its rank is 2.

Ideas for enhancing data structures

- **Use of Mangroves.**
- A red-black tree is a self-balancing binary lookup tree that ensures that the height of the tree is roughly $O(\log n)$, where n is the number of nodes in the tree.

- This property of the red-black tree allows it to maintain good performance in insertion, deletion and lookup operations.
- **Store subtree size.**
- In each node, in addition to storing the key value (key), the subtree size (size) of that node is also stored.
- Subtree size is the number of nodes in the subtree rooted at this node.
- Storing the subtree size allows the rank of an element to be quickly computed or the i-th smallest element to be found when performing sequential statistical operations.

Node representation

- A representation of the node is shown in the slide, which contains two main parts:
 - **key:** The key value of the node, i.e. the actual data stored in the node.
 - **size:** The size of the subtree rooted at this node.
- With this enhancement, dynamic sequential statistics operations can be accomplished in logarithmic time, which is useful for application scenarios where such operations need to be performed frequently.

Operation:

operations	method description	time complexity
Find Minimum (min)	Traverse from the root node down the left subtree until the leftmost node, which is the minimum node.	$O(\log n)$
Find Maximum (max)	Traverse down the right subtree from the root node to the rightmost node, which is the maximum node.	$O(\log n)$
find i-th smallest element	root at current node, compare i to left_size left_size: - if $i = \text{left_size} + 1$, current node is target node - if $i \leq \text{left_size}$, recursive left_size - else recursive right_size i - left_size - 1	$O(\log n)$
find the i-th largest element	translates to find the $(n - i + 1)$ th smallest element, where n is the total number of nodes in the tree.	$O(\log n)$

Von Neumann Architecture (Von Neumann Architecture)

The Von Neumann Architecture is a model of the infrastructure of modern computer systems, proposed by mathematician John von Neumann (John von Neumann) in 1945. It defines the basic components of a computer and how it works. The architecture is explained in detail below, with the English terms of the relevant terms labeled.

I. The Five Basic Components of the Von Neumann

Architecture

The von Neumann architecture consists of the following five main parts:

Chinese Name	English Terminology	Functional Description	Examples of Common Devices (with English)
Operators	Arithmetic Logic Unit (ALU)	Performs a variety of arithmetic operations (e.g., addition, subtraction, multiplication, division) and logical operations (e.g., AND, OR OR, NOT)	Typically integrated within the Central Processing Unit (CPU)
Controller	Control Unit (CU)	Responsible for fetching instructions from memory, interpreting instructions and coordinating their execution, and controlling the orderly work of other components	Also integrated inside the CPU and working with the ALU
Memory	Memory / Main Memory	Used to store instructions and data, it is a high-speed storage area directly accessible by the CPU	- Main Memory: RAM (Random Access Memory) - ROM (Read-Only Memory, Read-Only Memory) - **Cache Memory (Cache Memory)
			Memory can be subdivided into main memory and secondary memory: main memory is directly accessed by the CPU; secondary memory is used for long-term data retention
Input Device	Input Device	Used to input information, commands, or control signals to a computer	- Keyboard Keyboard - Mouse Mouse - Camera Camera / Webcam - Microphone Microphone - Scanner Scanner
Output Device	Output Device	presents the results of computer processing in a human-understandable form, such as text, images, sound, etc.	- Monitor Monitor / Screen - Printer Printer - Sound Speaker - Headphone Headphone


```

| Memory |
| Program + Data Storage Area |
+-----+-----+
|
v

```

```

+-----+-----+

```

```

| Controller Control Unit |

```

```

| +-----+ |
| |Program Counter PC| |

```

```

+-----+-----+

```

```

|

```

```

v

```

```

+-----+-----+

```

```

| Arithmetic | Operators

```

```

| Logic Unit (ALU)

```

```

+-----+-----+

```

```

|

```

```

v

```

```

+-----+-----+

```

```

| Output | Output | Output | Output | Output | Output | Output | Output

```

```

+-----+-----+

```

```

..

```

V. Strengths and limitations

Pros:

- Simplified computer design structure for easy hardware implementation
- Unified memory model that allows programs to read and write data flexibly
- Become the infrastructure for the development of modern computers

Limitations (also known as the "von Neumann Bottleneck" Von Neumann Bottleneck):

- **Instructions and data share the same memory bus**, resulting in limited read speeds (instructions and data cannot be read at the same time)
- Instructions are executed sequentially, not suitable for parallel processing tasks
- Cache misses can seriously slow down execution

VI. Modern Development (Supplementary)

Although today's computer architectures are still based on von Neumann, many optimization techniques have been introduced in practice, for example:

- **Harvard Architecture:** instructions and data are stored and processed separately to alleviate bottlenecks
- **Pipelining:** multiple instruction stages processed in parallel to increase throughput rates
- **Cache Memory:** small cache inside the CPU to reduce access latency

VII. Summary comparison table

Components	English Terminology	Functional Description
-----	-----	-----
Operators	Arithmetic Logic Unit	Arithmetic Logic Operations
Controller	Control Unit	Command control, coordinates the work of various components
Memory	Memory	Store data and program instructions
Input Devices	Input Device	Data Input Interface
Output Devices	Output Device	Data Output Interface
Program Counter (PC)	point to next instruction address	
Instruction Cycle	Instruction Cycle	Fetch → Translate → Execute → Write Back
Instruction Cycle	Instruction Cycle	Instruction Cycle
Von Neumann Bottleneck	Von Neumann Bottleneck	Limited by the processing speed of the instruction/data sharing bus

Amortized Analysis (AA)

Amortization analysis is a method used to analyze the average performance** of a **series of operations**. Instead of focusing on the worst-case scenario of a single operation, it analyzes the **Amortized Cost** of an entire sequence of operations.

I. Aggregate Method (Aggregate Method)

Rationale:

Assuming that (n) operations are performed at a total cost of ($T(n)$), the amortized cost of each operation is:

$$\frac{T(n)}{n}$$

That is, all operations are analyzed as a whole and the average cost is calculated.

Features:

- Simple and intuitive;
- Differences between operations are not taken into account;
- For situations where the cost of operation is not changing significantly or changes are predictable.

Examples:

Most insertions in dynamic array insertion operations cost ($O(1)$), but when the array is expanded, the insertion cost is ($O(n)$).

The aggregation method obtains the amortized cost by analyzing the total cost of all insertion operations and dividing by the number of insertions.

II. Accounting Method

Rationale:

- Assign an Amortized Cost to each operation;
- The excess cost is saved as **Balance (Credit)** and used to pay for subsequent, more expensive operations;
- Guaranteed total amortized cost $\geq$ actual total cost.

Strategy:

- **Cheap operation:** Amortized cost $>$ actual cost, with the excess as a balance;
- **Expensive operation:** Amortized cost $<$ actual cost, subsidized by previously accumulated balances.

Features:

- Finer control of cost allocation;
- Suitable for scenarios where some operations are extremely costly, but can be "subsidized" by other operations.

Examples:

Allocate \$3 per insertion in the dynamic array as amortized cost:

- The actual insertion only cost \$1;
- The remaining \$2 is set aside as a balance for future expansion operations.

III. Potential Method (Potential Method)

Rationale:

Define a **Potential Function** (Φ) for the data structure that represents the "pre-stored resources" in the current state.

The amortized cost of the first (i) operation is:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Among them:

- (c_i) : the actual cost of the (i) operation;
- $(\Phi(D_i))$: potential energy after the (i) th operation;
- $(\Phi(D_{i-1}))$: potential energy before operation;
- $(\hat{c}_i)$: amortized cost.

Explanation:

- If the potential energy increases, it means that this operation prepares for the follow-up;
- If the potential energy decreases, it means that the previously accumulated resources were used up.

Features:

- Mathematics is formalized and expressed flexibly and rigorously;
- Ideal for analyzing data structures with frequent dynamic state changes.

IV. Comparison of three methods

Methods	English Name	Analysis Granularity	Whether to Assign Amortized Costs to Each Operation	Characteristics
Aggregate Method	Aggregate Method	All operations as a whole	No	Simple and intuitive for scenarios where the regularity of operation costs is obvious
Accounting Method	Accounting Method	per operation	yes	with balance mechanism, suitable for individual operation is more expensive

Potential Method Potential Method Every Operation Yes Modeling with Potential Functions, suitable for complex state data structures

Let the set of objects of a certain class be S , whose **information entropy lower bound** is the minimum number of bits required to store objects of that class, denoted $I(n)$.

Then a **succinct data structure** is in space complexity:

$I(n) + o(I(n))$ bits

Under this, data structures that support efficient operations (e.g., querying, modifying, inserting, etc.) are also available.

where $o(I(n))$ is a low-order function much smaller than $I(n)$, such as $\log n$ or $\log \log n$.

III. Typical examples

1. Bit Vector + Rank/Select

For a bitmap $B \in \{0, 1\}^n$ of length n , the succinct data structure can be in:

- Space: $n + o(n)$ bit
- Operation:
- **rank(i)**: query how many 1s there are in $B[1..i]$
- **select(k)**: query for the location of the k th 1 occurrence

This type of structure is widely used for text indexing (e.g., FM-index) and graph compression.

2. Balanced Parentheses

Used to represent tree or expression structures (e.g., XML/JSON structured trees) that can be supported in near-theoretical space:

- Find matching brackets (match subtree)
- Query Parent-Child Node Relationships
- Subtree size calculation and other operations

3. Succinct Trees

Used to store tree structures such as:

- The optimal encoding of an ordered tree of n nodes requires about $2n$ bit
- Using structures such as Jacobson/Farzan-Munro can support queries for parents, children, LCAs (Least Common Ancestors), etc. within $2n + o(n)$ bit, with a time complexity of $O(1)$ or $O(\log n)$

IV. Comparison with other concepts

Types of Data Structures	Space Usage	Operational Efficiency	Features
traditional structures (e.g., chained lists, trees)	large (often with lots of pointers)	fast	space-wasting
Compressed Data Structures	Extremely Small Space	Doesn't necessarily support queries	Mostly used for compressed storage

Types of Data Structures	Space Usage	Operational Efficiency	Features
Succinct Data Structure	Near-optimal (information-theoretic lower bound)	Fast (usually $O(1)$)	Both compressed and manipulable

V. Application scenarios

- Text compression and full-text indexing (e.g., FM-index)
- :: Large-scale graph structures (Web graphs, social network graphs)
- Genome storage and querying (e.g. BWT structure)
- Compressed representation of XML/JSON trees
- Compressed Boolean matrix

VI. Summary

Characteristics	Description
space complexity	approaching lower bound on theoretical information entropy
Operations	Support common query operations with low time complexity
Applications	Large Scale Data, High Compression Ratio Requirement Scenarios
Common Construction Methods	Bitmap + Auxiliary Structure based, Tree Encoding, etc.

Simple data structures strike a high balance between "**compressed storage + manipulability** ", and are a very important class of structures in modern computing.

heap-like notation for a binary tree

A **Heap-like notation for a binary tree** is a way to represent a **Complete Binary Tree** by an array (usually starting at subscript 1), which is widely used in heap structures, but can also be used for sequential storage representation of other binary trees.

I. Basic ideas

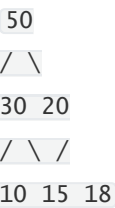
In a **heap-like notation**, the nodes of a binary tree are stored in an array in **Level-order traversal** order, numbered starting at **1** (or sometimes 0, but a typical heap is more intuitive starting at 1).

For a node numbered **i**, the relationship is as follows:

Relationships	Formulas	Meanings
Parent	$\lfloor i / 2 \rfloor$	The parent subscript of the i-th node. For example, if $i=4$, the parent is 2.
Left child node	$2 \times i$	The left child subscript of the i-th node. For example, if $i=2$, the left child is 4.
Right child node	$2 \times i + 1$	The right child subscript of the i-th node. For example, if $i=2$, the right child is 5.

II. Examples

Consider the following binary tree (with the largest heap as an example):



Stored as an array (starting at subscript 1) with heap-like notation:

subscript (i)	numeric
1	50
2	30
3	20
4	10
5	15
6	18

Relationship Validation:

- The left child of node 1 is at subscript 2 (30) and the right child is at subscript 3 (20)
- The left child of node 2 is at subscript 4 (10) and the right child is at subscript 5 (15)
- The left child of node 3 is at subscript 6 (18)

III. Advantages and limitations

✓ Advantages

- **No need for pointers**, implement tree structures with arrays
- For **complete binary trees** such as heaps
- **Parent-child relationships are computationally simple**, $O(1)$ Access

✗ Limitations

- Wasted array space if used for **non-complete binary trees**
- Unsuitable for representing highly unbalanced trees (e.g., chained structures)

IV. Application scenarios

- **Heap implementation**
- Maximum Heap / Minimum Heap
- priority queue
- **Heap Sort**
- **Segment Tree**
- **Fenwick Tree**
- for certain memory compact structures such as heap structures in graph algorithms (e.g. Dijkstra)

V. Comparison with other encoding methods

Encoding	Pointers or not	Space Efficiency	Applicable Structures
Heap-like notation	No	High	Complete/near-complete trees
Linked representations	Yes	Flexible	Arbitrary structures
Succinct representation	No (bit-coded)	Optimal	Large-scale static trees

Point Query: Min/Max, kth element access

Min-Max Heap

What is Min-Max Heap?

Min-Max Heap is an array-based Complete Binary Tree that divides nodes into **Min layers** and **Max layers** according to their depth, as follows:

- **The root node is at the Min level (depth 0)**, indicating the minimum value of the whole tree.
- **** Its child is at the Max level (depth 1)****, indicating the subtree maximum.

- **grandchild node back to the Min level (depth 2)**, denoting the subtree minimum.
- So alternately.

That is to say:

- **Even layers (0, 2, 4, ...)** are **Min layers** that maintain the minimum value;
- **Odd layers (1, 3, 5, ...)** are **Max layers** that maintain the maximum value.

Operation

operations	time complexity	description
<code>findMin()</code>	$O(1)$	root node
<code>findMax()</code>	$O(1)$	greater of two children of root
<code>insert(x)</code>	$O(\log n)$	inserts and "floats" to the correct position
<code>deleteMin()</code>	$O(\log n)$	delete root node and "sink"
<code>deleteMax()</code>	$O(\log n)$	delete max node and "sink"

1. Node Hierarchy Determination

Given a node's subscript `i` (starting from 1) in an array with layers:

`level(i) = floor(log2(i))`

Determines if it is at the Min level:

`is_min_level(i) = level(i) % 2 == 0`

2. Insertion operations

- Inserts the new element into the end of the array;
- Determine which layer (Min or Max) it is on;
- Compare and float up to the correct position, following the rules of Min Heap or Max Heap, respectively.

3. Delete operations (lookups are similar, except that there is no delete operation)

`deleteMin()`

- Delete the root node (subscript 1);
- Replace with the last element of the array;
- Sinks downward, maintaining the Min-Max Heap nature.

`deleteMax()`

- Find the larger of the two children (subscripts 2 and 3) of the root;
- Delete it and replace it with the last element;
- Downward "sinking".

Dynamic sequential statistics

Common operations

1. OS-SELECT(i, S).

- This operation returns the i -th smallest element in the dynamic set S .
- For example, if $i=3$, then the operation will return the third smallest element in the set.

2. OS-RANK(x, S).

- This operation returns the rank (i.e., the position) of the element x after it has been sorted in the set S .
- The rank is the position of the element in the sorted set, e.g., if x is the second smallest element in the set, then its rank is 2.

Ideas for enhancing data structures

- **Use of Mangroves.**
 - A red-black tree is a self-balancing binary lookup tree that ensures that the height of the tree is roughly $O(\log n)$, where n is the number of nodes in the tree.
 - This property of the red-black tree allows it to maintain good performance in insertion, deletion and lookup operations.
- **Store subtree size.**
 - In each node, in addition to storing the key value (key), the subtree size (size) of that node is also stored.
 - Subtree size is the number of nodes in the subtree rooted at this node.
 - Storing the subtree size allows the rank of an element to be quickly computed or the i -th smallest element to be found when performing sequential statistical operations.

Node representation

- A representation of the node is shown in the slide, which contains two main parts:
- **key:** The key value of the node, i.e. the actual data stored in the node.
- **size:** The size of the subtree rooted at this node.

With this enhancement, dynamic sequential statistics operations can be accomplished in logarithmic time, which is useful for application scenarios where such operations need to be performed frequently.

Operation:

operations	method description	time complexity
Find Minimum (min)	Traverse from the root node down the left subtree until the leftmost node, which is the minimum node.	$O(\log n)$
Find Maximum (max)	Traverse down the right subtree from the root node to the rightmost node, which is the maximum node.	$O(\log n)$

operations	method description	time complexity
find i-th smallest element	root at current node, compare i to left_size left_size: - if $i = \text{left_size} + 1$, current node is target node - if $i \leq \text{left_size}$, recursive left_size - else recursive right_size $i - \text{left_size} - 1$	$O(\log n)$
find the i-th largest element	translates to find the $(n - i + 1)$ th smallest element, where n is the total number of nodes in the tree.	$O(\log n)$

Von Neumann Architecture (Von Neumann Architecture)

The **Von Neumann Architecture** is a model of the infrastructure of modern computer systems, proposed by mathematician John von Neumann (**John von Neumann**) in 1945. It defines the basic components of a computer and how it works. The architecture is explained in **detail** below, with the English terms of the relevant terms labeled.

I. The Five Basic Components of the Von Neumann Architecture

The von Neumann architecture consists of the following five main parts:

Chinese Name	English Terminology	Functional Description	Examples of Common Devices (with English)
Operators	Arithmetic Logic Unit (ALU)	Performs a variety of arithmetic operations (e.g., addition, subtraction, multiplication, division) and logical operations (e.g., AND, OR OR, NOT)	Typically integrated within the Central Processing Unit (CPU)
Controller	Control Unit (CU)	Responsible for fetching instructions from memory, interpreting instructions and coordinating their execution, and controlling the orderly work of other components	Also integrated inside the CPU and working with the ALU
Memory	Memory / Main Memory	Used to store instructions and data, it is a high-speed storage area directly accessible by the CPU	- Main Memory: RAM (Random Access Memory) - ROM (Read-Only Memory, Read-Only Memory) - **Cache Memory (Cache Memory)

Chinese Name	English Terminology	Functional Description	Examples of Common Devices (with English)
			Memory can be subdivided into main memory and secondary memory: main memory is directly accessed by the CPU; secondary memory is used for long-term data retention
Input Device	Input Device	Used to input information, commands, or control signals to a computer	- Keyboard Keyboard - Mouse Mouse - Camera Camera / Webcam - Microphone Microphone - Scanner Scanner
Output Device	Output Device	presents the results of computer processing in a human-understandable form, such as text, images, sound, etc.	- Monitor Monitor / Screen - Printer Printer - Sound Speaker - Headphone Headphone

II. Core Ideas of the von Neumann System

1. Stored Program Control (Stored Program Concept)

Programs and data** are uniformly stored** in the main memory, which the computer reads and executes one by one in the order of instructions.

Prior to this, early computers required manual wiring or control panel changes to "change programs", which was highly inefficient.

2. Sequential Instruction Execution (Sequential Instruction Execution)

The CPU reads instructions from memory **in order** and automatically jumps to the next instruction after executing one, unless it encounters a jump instruction (such as `goto` or `if-else`).

3. Data and instructions are stored in the same memory (Unified Memory for Data and Instructions)

Program instructions and data** share the same storage area**, which both simplifies the hardware structure and creates potential problems (e.g., program code can be modified, protection mechanisms are needed).

III. Workflow: Instruction Cycle (Instruction Cycle)

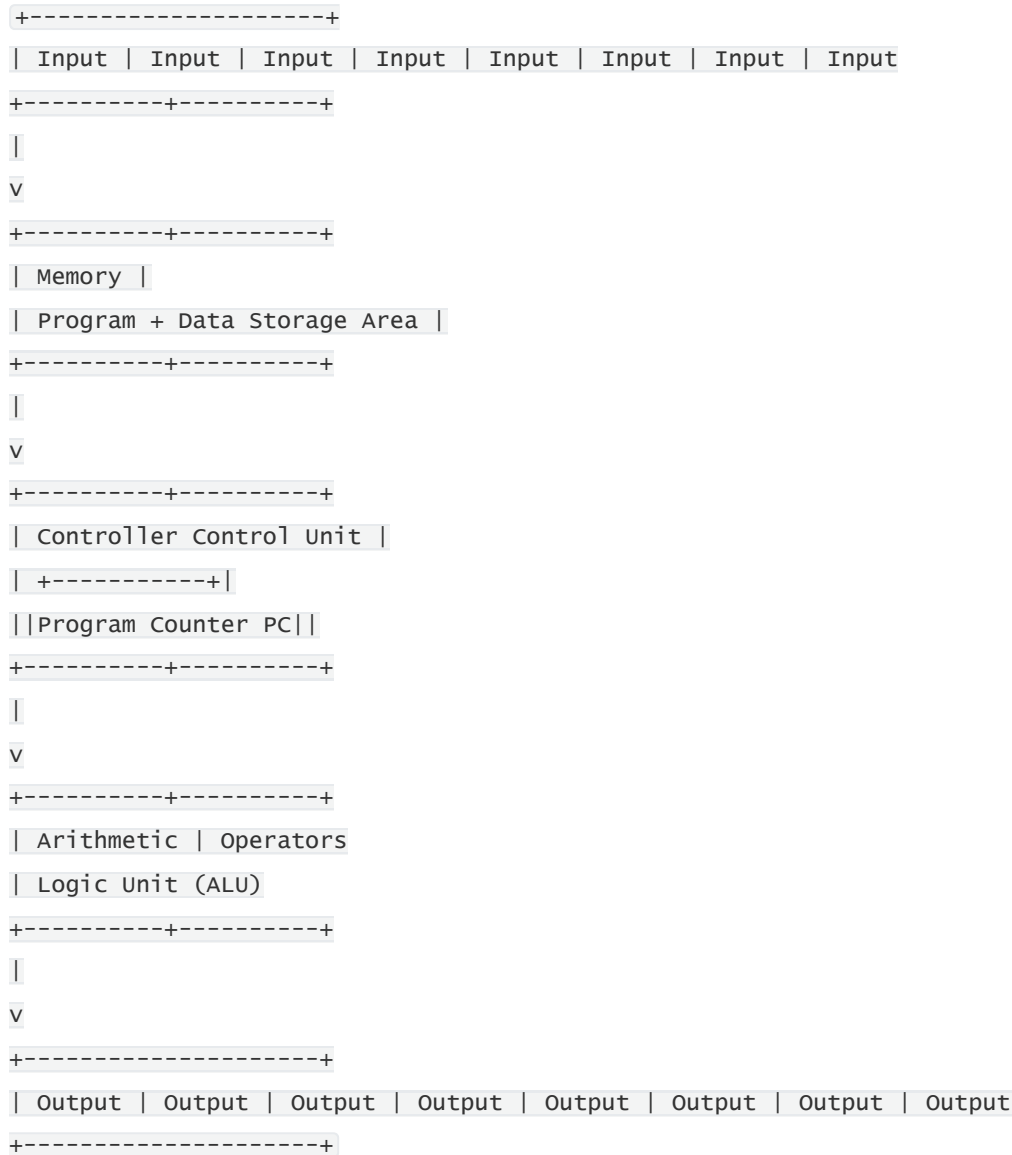
The execution of an instruction usually consists of the following steps:

1. **Fetch**: Fetch the next instruction from memory (at the address specified by the program counter PC).

2. **Decode:** The controller analyzes the instruction to determine the operation to be performed and the operand position.
3. **Execute:** An operator or other unit performs an instruction operation.
4. **Store/Writeback:** Write the result back to memory or registers.
5. **Update Program Counter (PC):** points to the address of the next instruction.

This process is called the **Fetch-Decode-Execute Cycle**.

IV. Schematic diagram of a typical structure (text version)



V. Strengths and limitations

Pros:

- Simplified computer design structure for easy hardware implementation
- Unified memory model that allows programs to read and write data flexibly
- Become the infrastructure for the development of modern computers

Limitations (also known as the "von Neumann Bottleneck" Von Neumann Bottleneck):

- **Instructions and data share the same memory bus**, resulting in limited read speeds (instructions and data cannot be read at the same time)
- Instructions are executed sequentially, not suitable for parallel processing tasks
- Cache misses can seriously slow down execution

VI. Modern Development (Supplementary)

Although today's computer architectures are still based on von Neumann, many optimization techniques have been introduced in practice, for example:

- **Harvard Architecture:** instructions and data are stored and processed separately to alleviate bottlenecks
- **Pipelining:** multiple instruction stages processed in parallel to increase throughput rates
- **Cache Memory:** small cache inside the CPU to reduce access latency

VII. Summary comparison table

Components	English Terminology	Functional Description
Operators	Arithmetic Logic Unit	Arithmetic Logic Operations
Controller	Control Unit	Command control, coordinates the work of various components
Memory	Memory	Store data and program instructions
Input Devices	Input Device	Data Input Interface
Output Devices	Output Device	Data Output Interface
Program Counter (PC)	point to next instruction address	
Instruction Cycle	Instruction Cycle	Fetch → Translate → Execute → Write Back
Von Neumann Bottleneck	Von Neumann Bottleneck	Limited by the processing speed of the instruction/data sharing bus

Amortized Analysis (AA)

Amortization analysis is a method used to analyze the average performance** of a **series of operations**. Instead of focusing on the worst-case scenario of a single operation, it analyzes the **Amortized Cost** of an entire sequence of operations.

I. Aggregate Method (Aggregate Method)

Rationale:

Assuming that (n) operations are performed at a total cost of ($T(n)$), the amortized cost of each operation is:

$$\frac{T(n)}{n}$$

That is, all operations are analyzed as a whole and the average cost is calculated.

Features:

- Simple and intuitive;
- Differences between operations are not taken into account;
- For situations where the cost of operation is not changing significantly or changes are predictable.

Examples:

Most insertions in dynamic array insertion operations cost ($O(1)$), but when the array is expanded, the insertion cost is ($O(n)$).

The aggregation method obtains the amortized cost by analyzing the total cost of all insertion operations and dividing by the number of insertions.

II. Accounting Method

Rationale:

- Assign an Amortized Cost to each operation;
- The excess cost is saved as **Balance (Credit)** and used to pay for subsequent, more expensive operations;
- Guaranteed total amortized cost $\geq$ actual total cost.

Strategy:

- **Cheap operation:** Amortized cost $>$ actual cost, with the excess as a balance;
- **Expensive operation:** Amortized cost $<$ actual cost, subsidized by previously accumulated balances.

Features:

- Finer control of cost allocation;
- Suitable for scenarios where some operations are extremely costly, but can be "subsidized" by other operations.

Examples:

Allocate \$3 per insertion in the dynamic array as amortized cost:

- The actual insertion only cost \$1;
- The remaining \$2 is set aside as a balance for future expansion operations.

III. Potential Method (Potential Method)

Rationale:

Define a **Potential Function** (Φ) for the data structure that represents the "pre-stored resources" in the current state.

The amortized cost of the first (i) operation is:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Among them:

- (c_i): the actual cost of the (i) operation;
- ($\Phi(D_i)$): potential energy after the (i) th operation;
- ($\Phi(D_{i-1})$): potential energy before operation;
- ($\hat{c}_i$): amortized cost.

Explanation:

- If the potential energy increases, it means that this operation prepares for the follow-up;
- If the potential energy decreases, it means that the previously accumulated resources were used up.

Features:

- Mathematics is formalized and expressed flexibly and rigorously;
- Ideal for analyzing data structures with frequent dynamic state changes.

IV. Comparison of three methods

Methods	English Name	Analysis Granularity	Whether to Assign Amortized Costs to Each Operation	Characteristics
Aggregate Method	Aggregate Method	All operations as a whole	No	Simple and intuitive for scenarios where the regularity of operation costs is obvious
Accounting Method	Accounting Method	per operation	yes	with balance mechanism, suitable for individual operation is more expensive

Methods	English Name	Analysis Granularity	Whether to Assign Amortized Costs to Each Operation	Characteristics
Potential Method	Potential Method	Every Operation	Yes	Modeling with Potential Functions, suitable for complex state data structures